1 Interpreted BAPI Services

1.1 Introduction

Interpreted Bapi services are web services that are converted by an interpreter to INSERT, UPDATE or DELETE SQL statements. Interpreted Bapi services enable writing access to tables and, as a result, have been designed for data maintenance. Along with interpreted Java services, you can completely implement maintenance and display of data via configurations.

The definition for the interpreter is created with the Repository Client and stored in XML files.

1.2 Features

1.2.1 Processing steps of the Bapi Interpreter

The Bapi Interpreter is structured in 5 steps. The steps are:

1.2.1.1 init

- has been designed for initialization: reads out input parameters; reads the service configuration
- **plausibility checks must not take place!**

1.2.1.2 checkKeys

- has been designed to check whether or not all key fields have been specified. All plausibility checks are performed in a row and then the client receives the result of the plausibility checks.

1.2.1.3 selectData

- selects data
  - data required for plausibility checks
  - data that are also required in performAction
  - etc.
- **plausibility checks must not take place!**

1.2.1.4 checkData

- All plausibility checks that do not pertain to the checking of key fields are performed here.

1.2.1.5 performAction

- the actual BAPI functions are executed here (e.g. create data record, lock data record, etc.)

1.2.2 Basic processing of the individual processing steps

1.2.2.1 init

- reads out the service configuration
- reads out request parameters

1.2.2.2 checkKeys

- checks whether at least one input parameter has been configured with the property "key field". The property "key field" is the constraint SERIAL if an input parameter including this constraint is available or it is the constraint KEY.
- checks whether at least 5 input parameters have been configured with the property "key field". The property "key field" is the constraint SERIAL if an input parameter including this constraint is available or it is the constraint KEY.
- checks whether all input parameters including the constraint KEY have a unique index between 1 and 5.
- checks whether all input parameters including the constraint KEY have been passed in the request
- checks whether all input parameters including the constraint SERIAL have been passed in the request

1.2.2.3 selectData

- selects a data record using the constraint SERIAL, if the data record and the constraint SERIAL are available
- selects data records using the constraint KEY, if data records and the constraint KEY are available
- reads internal locks
- reads external Locks

1.2.2.4 checkData

- checks whether all mandatory parameters have been transferred in the request (repository field "Is Mandatory“)

1.2.2.5 performAction

- no action, because specific to the function

1.2.3 Bapi functions

1.2.3.1 LOCK

The Bapi function LOCK has been designed to lock a data record for a later processing.

init

- basic processing only

checkKeys

- basic processing only

selectData

- basic processing only

checkData

- Basic processing
- checks if exactly one data record is available matching the property "key field” (filter). The property "key field" is the constraint SERIAL if an input parameter including this constraint is available or it is the constraint KEY.

performAction

- Locks the data record, if it has not yet been locked by another user

1.2.3.2 UNLOCK

The Bapi function UNLOCK has been designed to unlock a data record after processing or once processing has been interrupted.

init

- basic processing only

checkKeys

- basic processing only

selectData

- basic processing only

checkData

- Basic processing
- checks if exactly one data record is available matching the property "key field” (filter). The property "key field" is the constraint SERIAL if an input parameter including this constraint is available or it is the constraint KEY.
- checks whether a separate lock is at all available

performAction

- unlocks the data record

1.2.3.3 INSERT

The Bapi function INSERT adds a data record to the database.

init

- basic processing only

checkKeys

- basic processing only

selectData

- basic processing only

checkData

- Basic processing
- checks that no data record could be identified using the constraint KEY

performAction

- adds the data record and identifies the serial if a result parameter has been configured using the constraint SERIAL

1.2.3.4 UPDATE

A data record is edited using the function UPDATE.

init

- basic processing only

checkKeys

- basic processing only

selectData

- basic processing only

checkData

- Basic processing
- checks if exactly one data record is available matching the property "key field" (filter). The property "key field" is the constraint SERIAL if an input parameter including this constraint is available or it is the constraint KEY.
- checks whether a separate lock is at all available

performAction

- UPDATEs the data record

1.2.3.5 DELETE

The Bapi function DELETE deletes a data record from the database.

init

- basic processing only

checkKeys

- basic processing only

selectData

- basic processing only

checkData

- basic processing
- checks if exactly one data record is available matching the property "key field" (filter). The property "key field" is the constraint SERIAL if an input parameter including this constraint is available or it is the constraint KEY.
- checks whether or not an internal lock is available
- checks whether or not an external lock is available

performAction

- deletes the data record

1.3 Service definition

An interpreted Bapi service is defined in the repository (further information on the required values and their meaning: see section "repository data").

The service domain can be exported as XML file, once the definition has been completed. The resulting file (the one relevant to the Interpreter) is the file **<Domain>.Configuration.xml.**

1.4 Storage in a server

The file is located on a MW30 server under jdir\MOC\<Instance>\listInterpreter\<Scope> or JHYDRADIR\MOC\<Instance>\listInterpreter\<Scope>. The scope can have one of the following values: **standard, custom** or **local**.

1.5 Repository data

1.5.1 Tab Services

Name	Meaning	Optional
Domain	The domain of the service (e.g. BOPerson)	
Name	The complete service name, i.e. Domain.Function (e.g. BOPerson.insert)	
Service Function	The function of the service (e.g. insert)	
Service Type	Service type - for interpreted BAPI services, fixed: **InterpretedBapiService**	
Description	Brief (internal) description of the service	

1.5.2 Tab ServiceParameter

Name	Meaning	Optional
Domain	The domain of the service (e.g. BOPerson)	
Service Function	The function of the service (e.g. insert)	
Service	The complete service name, i.e. Domain.Function (e.g. BOPerson.insert)	
Acronym	Acronyms (e.g. person.id) have to be unique within a service	
Web service type	The data type of the parameter (decimal, integer, string, boolean, datetime)	
DB table	The table that is used to select the value for the acronym	X (if the field is used in UE only)
DB field	The field that is used to select the value for the acronym	X (if the field is used in UE only)

DB Alias	The table alias for the table that is used to select the value for the acronym	X (if the field is used in UE only)
Can Equal	If the field is an input parameter, this option must be enabled.	X (only if Result or Constraint "MODIFY_TS“ or "MODIFY_BY“ or "CREATE_TS“ or "CREATE_BY“ is set)
IsResult	Specifies whether or not it is a Result	X (if not SERIAL)
IsSpecialParameter	Specifies whether or not the field is an input parameter	X (only if Result or Constraint "MODIFY_TS“ or "MODIFY_BY“ or "CREATE_TS“ or "CREATE_BY“ is set)
IsMandatory	Specifies whether or not the field is a mandatory field. Only reasonable with input parameters. Mandatory fields must be sent with the request and must not be null or empty.	X
Constraints	See separate section "Constraints“	X (if no constraint is required for the current acronym)
ConditionalFieldKey	If a name of a FeatureSet is entered, this acronym is only processed, if the FeatureSet is active.	

1.5.2.1 ServiceParameter: Constraints

Constraints are processing parameters that are structured as key with optional values. You use the pipe (|) character to separate two keys. You use the equal sign (=) to separate key and value. You use a semicolon to separate various values. The general structure is as follows: e.g. Key1=value;value;value|Key2|Key3=value|

The following constraints are available:

Constraint Key	**Constraint value(s)**	**Description**
KEY	Exactly one number ranging between 1 and 5. This number may only be used once within a service, i.e. different acronyms with the constraint KEY also must have different numbers.	Define field as key including key number for hyd_lock table

	The configuration of this constraint for a specific acronym must be identical for UPDATE, DELETE, LOCK and UNLOCK. Example: the acronym workplace.id includes the Constraint KEY=1 in the service <domain>.update. In this case, workplace.id also must include the Constraint KEY=1 in the services <domain>.delete, <domain>.lock and <domain>.unlock!	
SERIAL	none	The field is a SERIAL (or auto-increment). The database generates this value when creating a data record. ⇨ "isResult“ needs to be active for the INSERT service!
SEP_DATETIME	1st parameter refers to the date field 2nd parameter refers to the time field	Allows processing of separate date and time fields
BOOL	1st parameter refers to the value that is to be entered into the database, if true. 2nd parameter refers to the value that is to be entered in the database, if false. 3rd parameter refers to the value that is to be entered into the database, if null (“null” for writing the DB null -> default). 4th parameter refers to the type of the DB field. For example: BOOL=J;N;null;string| BOOL=1;0;null;integer|	Allows for Boolean values to be written in a string or integer field

MODIFY_TS	None	The acronym represents the point in time of processing. -> To do so, the point in time of web service processing in the server is written into the database. ⇨ **No** parameter value is used!
MODIFY_BY	None	The acronym represents the user. The registered user executing the service is entered in the database. ⇨ **No** parameter value is used!
CREATE_TS	None	The acronym represents the point in time of generation -> To do so, the point in time of web service processing in the server is written into the database. ⇨ **No** parameter value is used!
CREATE_BY	None	The acronym represents the user who created the data record. The registered user executing the service is entered in the database. ⇨ **No** parameter value is used!
CONST_VALUE	String value for the constant e.g. CONST_VALUE=J	You can create constant values for database fields with this constraint. The parameter must be in the repository. The parameter must not be isSpecialParameter=Y or isResult =Y!

SUB_STR	. 1st parameter is the database field (without alias). 2nd parameter is the position in the database field. 3rd parameter is the parameter length in the database field. e.g.: param1;1;1	You can use this constraint to map the content of a database field using several acronyms. For writing, you must always pass all acronyms for this DB field at the same time. If this is not the case, a plausibility error is thrown that includes all acronyms for this DB field. You can easily combine this constraint with CONST_VALUE and BOOL.
STR_VALUE_RESTRICTION	List of valid values separated by semicolon ";". To allow NULL or empty string (are processed the same way), include an empty string between two semicolons ";". To allow a semicolon ";" within a valid value, you must mask it with a backslash "\". To allow a backslash "\" within a valid value, you must mask it with another backslash "\\". e.g.: NULL and empty string and J and N are allowed values: STR_VALUE_RESTRICTION=J;N;; e.g.: Semicolon an J are allowed values: STR_VALUE_RESTRICTION=\;;J e.g.:	You can use this constraint to implement value restrictions without exit for string parameters. If the parameter value is not included in the list of allowed values, a plausibility error is thrown.

	The values Foo and Bar are allowed values: STR_VALUE_RESTRICTION=Foo;Bar	
LOCK_KEY_ONLY	None	Using this constraint and the constraint CONST_VALUE, you can enter constants for the lock key that otherwise are not persisted in a DB field. You can also add this constraint to parameters. These parameters are then only stored in the lock table and not in a DB field.
SKIP_INTERPRETER	None	Use this constraint to tag fields that are only processed in exits and not by the interpreter.
LOGICAL_KEY	None	Only available for UPDATE and INSERT: You can use this constraint to tag logical keys. This only makes sense, if the technical key is a SERIAL. You can use the constraint LOGICAL_KEY to issue a plausibility error message. This error message is issued, if you try to create a new data record with identical logical keys or if you try to change an existing data record using the same logical keys that are used by an existing data record.

The database table of an interpreted Bapi service is **exclusively** taken from the first (first from the top) acronym configuration of a service that includes the constraint SERIAL or KEY. In this context, the constraint SERIAL takes priority, if it is available.

1.6 User exits

User exits provide entry points to enable changes to the configured behavior via programming.

Instead of the user exits and program exits presented below, use the GlobalExits. The GlobalExits ensure the greatest possible compatibility in the further development of the system, as they are supported equally for all service types.

1.6.1 Available user exits

1.6.1.1 sdiAfterInit

Via this user exit, the application developer can use the interpreted Bapi service to change (insert, delete, change) the read request parameters.

You must **NOT** perform a plausibility check. You must perform plausibility checks in sdiAfterCheckData. The results can then be added to the result of sdiAfterCheckData.

1.6.1.2 sdiAfterSelectData

Via this user exit, the application developer can use the interpreted Bapi service to select data, e.g. for a plausibility check.

You must **NOT** perform a plausibility check. You must perform plausibility checks in sdiAfterCheckData. The results can then be added to the result of sdiAfterCheckData.

1.6.1.3 sdiAfterCheckData

Via this user exit, an application developer can create plausibility checks.

You must **NOT** perform an SQL query. You **must** perform SQL queries in sdiAfterSelectData. You can then use the results in this user exit.

You must **NOT** use an exception to tag a plausibility error here. For each plausibility error, you must create an object BapiInterpreterValidationMessage.

1.6.1.4 sdiAfterPerformAction

Via this user exit, the application developer can use the interpreted Bapi service to perform actions after the actual Bapi processing.

1.6.1.5 sdiBapiCleanup

Via this user exit, the application developer can use the interpreted Bapi service to perform cleanup actions after the actual Bapi processing. This exit is always executed, whether or not errors occurred.

1.6.2 Specifications for the implementation class

Package name: You must include the class in a package that consists of the domain name (in lower case letters). Further subpackages are **not** allowed.

Example: The service is called "MDUserAccountRules.insert“, consequently the package is called "mduseraccountrules“

Class name: The class must have a name of the following structure: domain name in lower case letters, whereas the first letter is written in capital letters, the name of the service function follows and is written in lower case letters, whereas the first letter is once again written in upper case.

Example: The service is called "MDUserAccountRules.insert“, consequently the class is called "MduseraccountrulesInsert“

The following definition applies for customized class names:
Customized names include “_“ (see naming conventions)
After ”_“ the first letter is changed to upper case and the underscore is skipped

Example: U_CUST_Units_sample.insert => UCustUnitsSampleInsert

Implemented interfaces / methods: no specifications

Other: The class must have a default constructor without parameters

Compilation: The Jar files *MpdvDomCoreSdiCompileLib.jar* and *MpdvDomCoreUserExitCompileLib.jar* must be included in the class path for the compile process.

Deployment: The class file of the user exit must be stored in the directory jdir/MOC/<instance>/userexit/<scope> or <JHYDRADIR>/MOC/<instance>/userexit/<scope> including package directory structure.

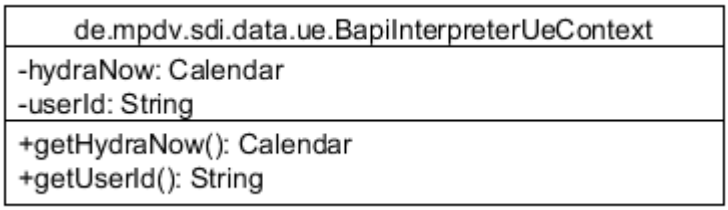
Example: User exit sdiAfterCheckData for service "MDUserAccountRules.insert":

```
package mduseraccountrules;

import de.mpdv.customization.userExit.IUserExitParam;

/**
 * Sample user exit
 *
 *
 */
public class MduseraccountrulesInsert
{

    public void sdiAfterCheckData(final IUserExitParam param)
    {
        // TODO implementation
    }
}
```

Directory structure on the server (instance 1, scope custom):
jdir/MOC/1/userexit/custom/mduseraccountrules/MduseraccountrulesInsert.class

1.6.3 Interfaces

1.6.3.1 Class: BapiInterpreterUeContext



This context class provides data that can be used globally in interpreted Bapi services in the context of user exits.

Field	Description

hydraNow	The property "hydraNow" is a time stamp created at the beginning of web service processing and, as a result, can be used as reference time stamp for the current web service call.
userId	Includes the user logged on to the client.

1.6.3.2 Userexit: sdiAfterInit

Parameter key in IUserExitParam:

Key name	Type	Description
param	SdiAfterInitParam	Parameter structure for the user exit
context	BapiInterpreterUeContext	Context structure for all user exits of the interpreted Bapi services
factory	ISystemUtilFactory	Utility class to access system utilities, such as logger or DB connection in user exits.

Return key in IUserExitParam:

Key name	Type	Description
result	SdiAfterInitResult	Result structure for the user exit: parameter structure including the changes made in the user exit

1.6.3.3 User exit: sdiAfterSelectData

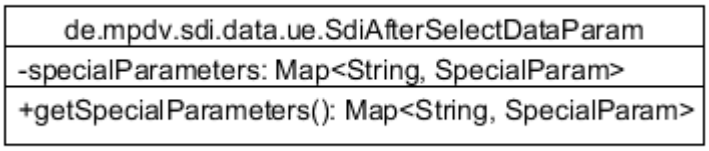
Parameter key in IUserExitParam:

Key name	Type	Description
param	SdiAfterSelectDataParam	Parameter structure for the user exit
context	BapiInterpreterUeContext	Context structure for all user exits of the interpreted Bapi services
factory	ISystemUtilFactory	Utility class to access system utilities, such as logger or DB connection in user exits.

Return key in IUserExitParam:

No return available

Class diagram of the parameter structure:



1.6.3.4 Userexit: sdiAfterCheckData

Parameter key in IUserExitParam:

Key name	Type	Description
param	SdiAfterCheckDataParam	Parameter structure for the user exit
context	BapiInterpreterUeContext	Context structure for all user exits of the interpreted Bapi services
factory	ISystemUtilFactory	Utility class to access system utilities, such as logger or DB connection in user exits.

Return key in IUserExitParam:

Key name	Type	Description
result	SdiAfterCheckDataResult	Result structure for the user exit: includes plausibility errors

1.6.3.5 Userexit: sdiAfterPerformAction

Parameter key in IUserExitParam:

Key name	Type	Description
param	SdiAfterPerformActionParam	Parameter structure for the user exit
context	BapiInterpreterUeContext	Context structure for all user exits of the interpreted Bapi services

factory	ISystemUtilFactory	Utility class to access system utilities, such as logger or DB connection in user exits.

Return key in IUserExitParam:

No return available

1.6.3.6 Userexit: sdiBapiCleanup

Parameter key in IUserExitParam:

Key name	Type	Description
param	SdiBapiCleanupParam	Parameter structure for the user exit
context	BapiInterpreterUeContext	Context structure for all user exits of the interpreted Bapi services
factory	ISystemUtilFactory	Utility class to access system utilities, such as logger or DB connection in user exits.

Return key in IUserExitParam:

No return available

1.6.3.7 Class SdiAfterInitParam

Field	Type	Description
specialParameters	Map<String, SpecialParam>	Assigns acronym => SpecialParameter for all web service parameters of the type "is SpecialParameter“

1.6.3.8 Class SdiAfterInitResult

Field	Type	Description

specialParameters	Map<String, SpecialParam>	Assigns acronym => SpecialParameter for all web service parameters of the type "is SpecialParameter“

1.6.3.9 Class SdiAfterSelectDataParam

Field	Type	Description
specialParameters	Map<String, SpecialParam>	Assigns acronym => SpecialParameter for all web service parameters of the type "is SpecialParameter“

1.6.3.10 Class SdiAfterCheckDataParam

Field	Type	Description
specialParameters	Map<String, SpecialParam>	Assigns acronym => SpecialParameter for all web service parameters of the type "is SpecialParameter“
originalRecord	IBapiInterpreterDbRecord	DB record that includes all fields of the DB table that must be maintained (select * from <table>)
mergedRecord	IBapiInterpreterDbRecord	DB record that includes all fields of originalRecord and that was overwritten by all changes to the current data record.

1.6.3.11 Class SdiAfterCheckDataResult

Field	Type	Description
validationMessages	List<BapiInterpreterValidationMessage>	List of all validation/plausibility errors from the user exit

1.6.3.12 Class BapiInterpreterValidationMessage

Instances of this class represent plausibility errors specific to applications.

Field	Type	Description
languageKey	String	Language key that can be translated.
parameters	List<String>	Parameter values matching the language key

1.6.3.13 Class SdiAfterPerformActionParam

Field	Type	Description
specialParameters	Map<String, SpecialParam>	Assigns acronym => SpecialParameter for all web service parameters of the type "is SpecialParameter“
createdSerial	Integer	The created Serial (only with function insert, if a serial column is available for the BAPI)

1.6.3.14 Interface IBapiInterpreterDbRecord

Method	Description
fetchValueOfDbField(String dbField): Object	Fetches the DB value for the specified DB field. If the value does not exist or is NULL, NULL is returned.

1.6.3.15 Class SdiBapiCleanupParam

This class is "for future use" and does currently not include any data.

Field	Type	Description